

AI Knowledge Workflow Assistant

Full-stack RAG application documentation

A full-stack RAG application for asking grounded questions over uploaded documents. The app combines a FastAPI backend, LangGraph workflow orchestration, PostgreSQL persistence, Chroma vector search, a Streamlit frontend, Docker, and GitHub Actions CI.

The default mode is inexpensive: local deterministic embeddings and a local answer formatter are used unless OpenAI generation is explicitly enabled.

Highlights

- Upload text or Markdown documents through an API or Streamlit UI
- Chunk and store document metadata in PostgreSQL
- Index chunks in Chroma for semantic retrieval
- Answer questions with citations from retrieved chunks
- Orchestrate the RAG flow with LangGraph
- Persist conversations and view saved history
- Run the full stack locally with Docker Compose
- Validate changes with automated tests in GitHub Actions

Tech Stack

- Python 3.12
- FastAPI
- LangGraph
- LangChain Core
- OpenAI API, optional
- PostgreSQL
- Chroma
- Streamlit
- Docker and Docker Compose
- Pytest
- GitHub Actions

Architecture

```

Streamlit UI
|
FastAPI API
|
|-- PostgreSQL: documents, chunks, conversations, messages
|-- Chroma: vector index for retrieval
|-- LangGraph: retrieve -> route -> answer/fallback -> persist
|
Answer provider
|-- Local formatter by default
|-- OpenAI provider when enabled

```

Demo Flow

Use this flow when showing the project:

1. Start the Docker stack.
2. Open the Streamlit frontend.
3. Click Load sample knowledge base in the sidebar.
4. Choose one of the example questions above the ask box.
5. Submit the question and review the answer with citations.
6. Open the history selector in the sidebar to show saved conversations.

This demonstrates ingestion, vector retrieval, LangGraph orchestration, conversation persistence, citations, and the user interface without requiring paid API calls.

Run With Docker

Create a local .env file from the example values:

```
cp .env.example .env
```

Start the API, frontend, and PostgreSQL:

```
docker compose up --build
```

Open the frontend:

```
http://127.0.0.1:8501
```

API health check:

```
http://127.0.0.1:8001/health
```

Stop the stack:

```
docker compose down
```

Run Locally

Activate the virtual environment and start the API:

```
source .venv/bin/activate
uvicorn app.main:app --reload
```

Run the frontend in a second terminal:

```
source .venv/bin/activate
API_BASE_URL=http://127.0.0.1:8000 streamlit run frontend/app.py
```

API Examples

Upload a text or Markdown document:

```
curl -X POST http://127.0.0.1:8001/documents \
-F "file=@README.md;type=text/markdown"
```

List uploaded documents:

```
curl http://127.0.0.1:8001/documents
```

Search indexed chunks:

```
curl -X POST http://127.0.0.1:8001/documents/search \
-H "Content-Type: application/json" \
-d '{"query":"database migrations", "limit": 3}'
```

Ask a retrieval-grounded question:

```
curl -X POST http://127.0.0.1:8001/query \
  -H "Content-Type: application/json" \
  -d '{"question": "What does this project use for database migrations?", "limit": 3}'
```

View saved conversations:

```
curl http://127.0.0.1:8001/conversations
```

OpenAI Mode

By default, answers use a free local formatter. To use OpenAI for answer generation, set these values in `.env`:

```
ANSWER_PROVIDER="openai"
OPENAI_API_KEY="your-api-key"
OPENAI_ANSWER_MODEL="gpt-5.5-mini"
```

LangSmith Tracing

LangGraph runs can be traced to LangSmith so you can inspect the RAG workflow on the LangChain/LangSmith website. Add these values to `.env`, restart the API, then run a query:

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY="your-langsmith-api-key"
LANGSMITH_PROJECT="ai-knowledge-workflow-assistant"
```

Open <https://smith.langchain.com/> and select the matching project. Each query records named spans for retrieval, answer generation, fallback handling, and conversation persistence.

Development

Run tests:

```
pytest
```

Apply database migrations:

```
alembic upgrade head
```

The same test suite runs in GitHub Actions on pushes and pull requests to main.

Resume Summary

Built a Dockerized, full-stack RAG assistant with FastAPI, LangGraph, PostgreSQL, Chroma, Streamlit, automated tests, conversation persistence, citations, optional OpenAI generation, and GitHub Actions CI.